

Department of Electronics and Telecommunication Engineering
(NBA ACCREDITED)
Digital System Design Laboratory
Academic Year 2025-2026, SEM-5, Odd Semester

Course Code	EC24P
Subject Professor In-charge	Prof. Satendra Mane
Student Name	Rahul Bhadane
Roll Number	23104B003
Class	T.E – EXTC/EXCS SEM-V
Division	B
Date of Performance	18-7-25
Date of Submission	25-7-25

EXPERIMENT NO.1

Estimated time to complete this experiment: 02 hours.

Objective: To simulate and Implement all the Basic Gates using all methods of modeling.

CO to be achieved: CO1, CO2

Expected Outcome of Experiment:

- Students will learn VIVADO Design tool flow.
- Students will learn the basics of SPARTAN-7 FPGA Board.
- Students will learn how to simulate and synthesis the basic digital circuits in VIVADO.

Pre Lab/ Prior Concept: Knowledge of Programmable logic devices.

Apparatus/TOOL:

- PC
- VIVADO TOOL
- SPARTAN-7 FPGA Board.

VERILOG CODE.

USING GATE LEVEL MODELLING.

1. AND Gate

AND Gate Code.

```
module AND_Gate(output Y, input A, B);  
    and (Y, A, B);  
endmodule
```

Test Bench for AND Gate.

```
module AND_Gate_tb;  
    reg A, B;  
    wire Y;  
  
    // Instantiate the AND gate module  
    AND_Gate uut (.Y(Y), .A(A), .B(B));  
  
    initial begin  
        // Test cases  
        $monitor("A = %b, B = %b, Y = %b", A, B, Y);  
        A = 0; B = 0; #10;  
        A = 0; B = 1; #10;  
        A = 1; B = 0; #10;  
        A = 1; B = 1; #10;  
        $stop;  
    end  
endmodule
```

2. OR Gate

OR Gate Code

```
module OR_Gate(output Y, input A, B);  
    or (Y, A, B);  
endmodule
```

Test Bench for OR Gate.

```
module OR_Gate_tb;  
    reg A, B;  
    wire Y;  
  
    // Instantiate the OR gate module  
    OR_Gate uut (.Y(Y), .A(A), .B(B));
```

```
initial begin
// Test cases
$monitor("A = %b, B = %b, Y = %b", A, B, Y);
A = 0; B = 0; #10;
A = 0; B = 1; #10;
A = 1; B = 0; #10;
A = 1; B = 1; #10;
$stop;
end
endmodule
```

3. XOR Gate

XOR Gate Code.

```
module XOR_Gate(output Y, input A, B);
xor (Y, A, B);
endmodule
```

Test Bench for XOR Gate.

```
module XOR_Gate_tb;
reg A, B;
wire Y;

// Instantiate the XOR gate module
XOR_Gate uut (.Y(Y), .A(A), .B(B));

initial begin
// Test cases
$monitor("A = %b, B = %b, Y = %b", A, B, Y);
A = 0; B = 0; #10;
A = 0; B = 1; #10;
A = 1; B = 0; #10;
A = 1; B = 1; #10;
$stop;
end
endmodule
```

4. XNOR Gate

XNOR Gate Code.

```
module XNOR_Gate(output Y, input A, B);
xnor (Y, A, B);
endmodule
```

Test Bench for XNOR Gate.

```
module XNOR_Gate_tb;
```

```
reg A, B;  
wire Y;  
  
// Instantiate the XNOR gate module  
XNOR_Gate uut (.Y(Y), .A(A), .B(B));  
  
initial begin  
    // Test cases  
    $monitor("A = %b, B = %b, Y = %b", A, B, Y);  
    A = 0; B = 0; #10;  
    A = 0; B = 1; #10;  
    A = 1; B = 0; #10;  
    A = 1; B = 1; #10;  
    $stop;  
end  
endmodule
```

5. NOT Gate

NOT Gate Code.

```
module NOT_Gate(output Y, input A);  
    not (Y, A);  
endmodule
```

Test Bench for NOT Gate.

```
module NOT_Gate_tb;  
    reg A;  
    wire Y;  
  
    // Instantiate the NOT gate module  
    NOT_Gate uut (.Y(Y), .A(A));  
  
    initial begin  
        // Test cases  
        $monitor("A = %b, Y = %b", A, Y);  
        A = 0; #10;  
        A = 1; #10;  
        $stop;  
    end  
endmodule
```

USING DATA FLOW MODELLING.

1. AND Gate

AND Gate Code.

```
module AND_Gate(output Y, input A, B);  
    assign Y = A & B; // Dataflow modeling  
endmodule
```

Test Bench for AND Gate.

```
module AND_Gate_tb;  
    reg A, B;  
    wire Y;  
  
    // Instantiate the AND gate module  
    AND_Gate uut (.Y(Y), .A(A), .B(B));  
  
    initial begin  
        // Test cases  
        $monitor("A = %b, B = %b, Y = %b", A, B, Y);  
        A = 0; B = 0; #10;  
        A = 0; B = 1; #10;  
        A = 1; B = 0; #10;  
        A = 1; B = 1; #10;  
        $stop;  
    end  
endmodule
```

2. OR Gate

OR Gate Code.

```
module OR_Gate(output Y, input A, B);  
    assign Y = A | B; // Dataflow modeling  
endmodule
```

Test Bench for OR Gate.

```
module OR_Gate_tb;  
    reg A, B;  
    wire Y;  
  
    // Instantiate the OR gate module  
    OR_Gate uut (.Y(Y), .A(A), .B(B));  
  
    initial begin  
        // Test cases
```

```
$monitor("A = %b, B = %b, Y = %b", A, B, Y);  
A = 0; B = 0; #10;  
A = 0; B = 1; #10;  
A = 1; B = 0; #10;  
A = 1; B = 1; #10;  
$stop;  
end  
endmodule
```

3. XOR Gate

XOR Gate Code.

```
module XOR_Gate(output Y, input A, B);  
    assign Y = A ^ B; // Dataflow modeling  
endmodule
```

Test Bench for XOR Gate.

```
module XOR_Gate_tb;  
    reg A, B;  
    wire Y;  
  
    // Instantiate the XOR gate module  
    XOR_Gate uut (.Y(Y), .A(A), .B(B));  
  
    initial begin  
        // Test cases  
        $monitor("A = %b, B = %b, Y = %b", A, B, Y);  
        A = 0; B = 0; #10;  
        A = 0; B = 1; #10;  
        A = 1; B = 0; #10;  
        A = 1; B = 1; #10;  
        $stop;  
    end  
endmodule
```

4. XNOR Gate

XNOR Gate Code.

```
module XNOR_Gate(output Y, input A, B);  
    assign Y = ~(A ^ B); // Dataflow modeling  
endmodule
```

Test Bench for XNOR Gate.

```
module XNOR_Gate_tb;  
    reg A, B;  
    wire Y;
```

```
// Instantiate the XNOR gate module
XNOR_Gate uut (.Y(Y), .A(A), .B(B));

initial begin
    // Test cases
    $monitor("A = %b, B = %b, Y = %b", A, B, Y);
    A = 0; B = 0; #10;
    A = 0; B = 1; #10;
    A = 1; B = 0; #10;
    A = 1; B = 1; #10;
    $stop;
end
endmodule
```

5. NOT Gate

NOT Gate Code.

```
module NOT_Gate(output Y, input A);
    assign Y = ~A; // Dataflow modeling
endmodule
```

Test Bench for NOT Gate.

```
module NOT_Gate_tb;
    reg A;
    wire Y;

    // Instantiate the NOT gate module
    NOT_Gate uut (.Y(Y), .A(A));

    initial begin
        // Test cases
        $monitor("A = %b, Y = %b", A, Y);
        A = 0; #10;
        A = 1; #10;
        $stop;
    end
endmodule
```

USING BEHAVIORAL MODELLING.

1. AND Gate

AND Gate Code.

```
module AND_Gate(output reg Y, input A, B);
    always @(*) begin
        Y = A & B; // Behavioral modeling
    end
end
```

endmodule

Test Bench for AND Gate.

```
module AND_Gate_tb;
    reg A, B;
    wire Y;

    // Instantiate the AND gate module
    AND_Gate uut (.Y(Y), .A(A), .B(B));

    initial begin
        $monitor("A = %b, B = %b, Y = %b", A, B, Y);
        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;
        $stop;
    end
endmodule
```

2. OR Gate

OR Gate Code.

```
module OR_Gate(output reg Y, input A, B);
    always @(*) begin
        Y = A | B; // Behavioral modeling
    end
endmodule
```

Test Bench for OR Gate.

```
module OR_Gate_tb;
    reg A, B;
    wire Y;

    // Instantiate the OR gate module
    OR_Gate uut (.Y(Y), .A(A), .B(B));

    initial begin
        $monitor("A = %b, B = %b, Y = %b", A, B, Y);
        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;
        $stop;
    end
endmodule
```

3. XOR Gate

Department of Electronics and Telecommunication Engineering
(NBA ACCREDITED)
Digital System Design Laboratory
Academic Year 2025-2026, SEM-5, Odd Semester

XOR Gate Code.

```
module XOR_Gate(output reg Y, input A, B);  
  always @(*) begin  
    Y = A ^ B; // Behavioral modeling  
  end  
endmodule
```

Test Bench for XOR Gate.

```
module XOR_Gate_tb;  
  reg A, B;  
  wire Y;  
  
  // Instantiate the XOR gate module  
  XOR_Gate uut (.Y(Y), .A(A), .B(B));  
  
  initial begin  
    $monitor("A = %b, B = %b, Y = %b", A, B, Y);  
    A = 0; B = 0; #10;  
    A = 0; B = 1; #10;  
    A = 1; B = 0; #10;  
    A = 1; B = 1; #10;  
    $stop;  
  end  
endmodule
```

4. XNOR Gate

XNOR Gate Code.

```
module XNOR_Gate(output reg Y, input A, B);  
  always @(*) begin  
    Y = ~(A ^ B); // Behavioral modeling  
  end  
endmodule
```

Test Bench for XNOR Gate.

```
module XNOR_Gate_tb;  
  reg A, B;  
  wire Y;  
  
  // Instantiate the XNOR gate module  
  XNOR_Gate uut (.Y(Y), .A(A), .B(B));  
  
  initial begin  
    $monitor("A = %b, B = %b, Y = %b", A, B, Y);  
    A = 0; B = 0; #10;  
    A = 0; B = 1; #10;  
    A = 1; B = 0; #10;  
    A = 1; B = 1; #10;  
  end  
endmodule
```

```
$stop;
end
endmodule
```

5. NOT Gate

NOT Gate Code.

```
module NOT_Gate(output reg Y, input A);
  always @(*) begin
    Y = ~A; // Behavioral modeling
  end
endmodule
```

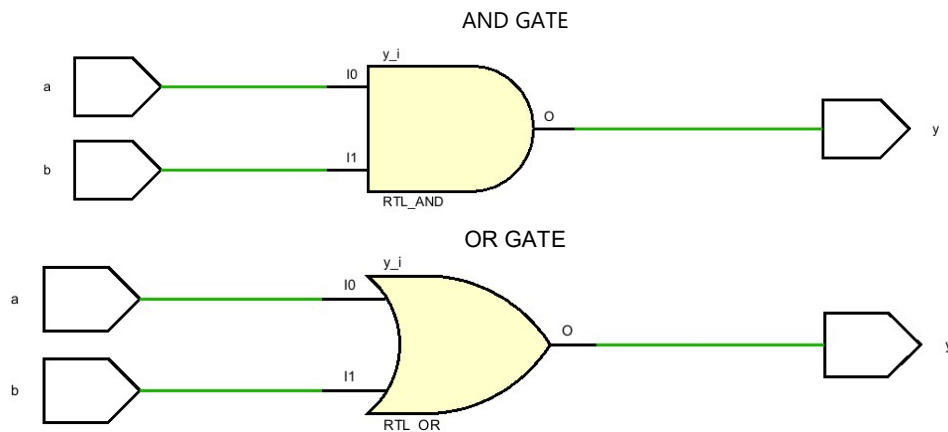
Test Bench for NOT Gate

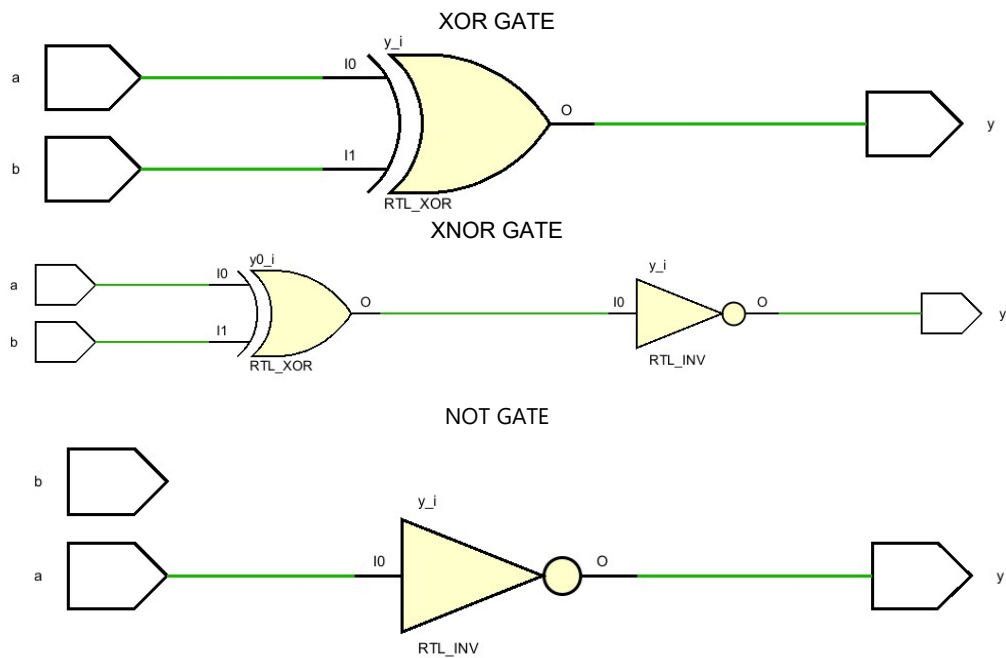
```
verilog
CopyEdit
module NOT_Gate_tb;
  reg A;
  wire Y;

  // Instantiate the NOT gate module
  NOT_Gate uut (.Y(Y), .A(A));

  initial begin
    $monitor("A = %b, Y = %b", A, Y);
    A = 0; #10;
    A = 1; #10;
    $stop;
  end
endmodule
```

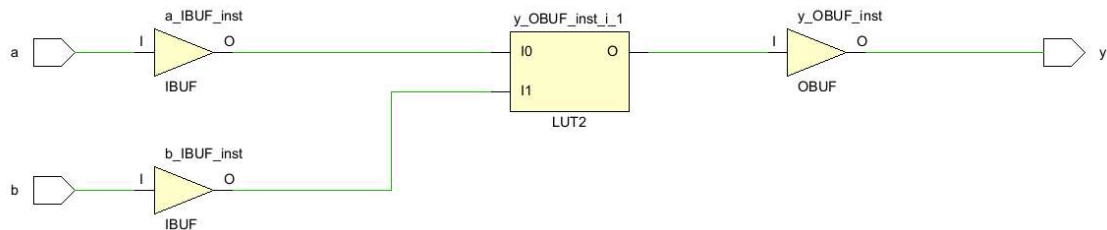
SIMULATION RESULTS.



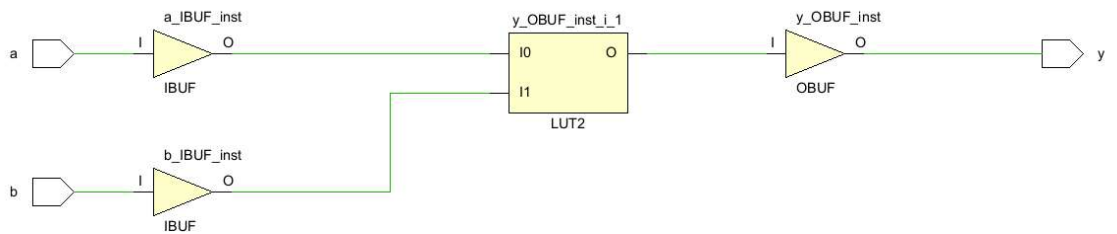


SYNTHESIS DIAGRAM.

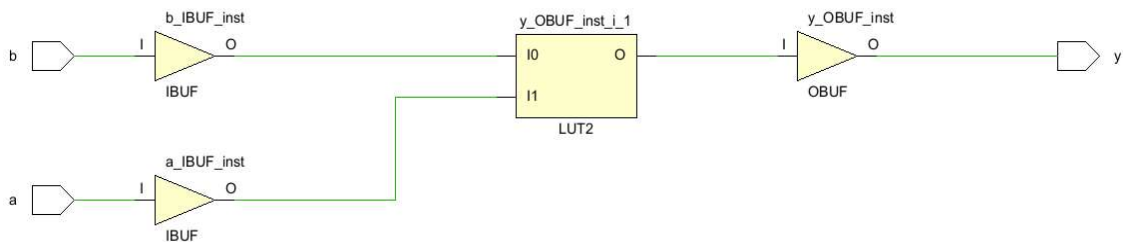
AND GATE



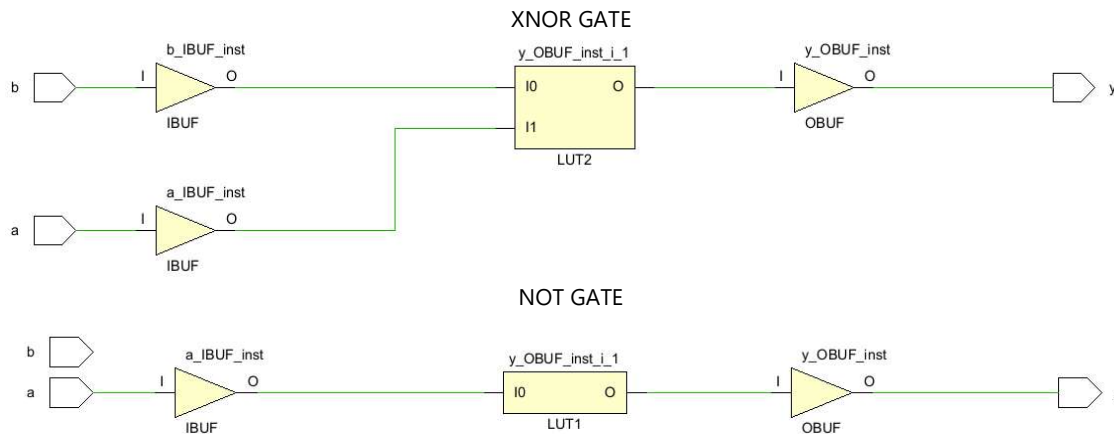
OR GATE



XOR GATE



**Department of Electronics and Telecommunication Engineering
(NBA ACCREDITED)
Digital System Design Laboratory
Academic Year 2025-2026, SEM-5, Odd Semester**



CONCLUSION.

In this experiment, we successfully simulated and implemented the basic logic gates (AND, OR, XOR, XNOR, NOT) using different modeling methods on the Spartan-7 FPGA board. You learned the fundamental principles of digital logic design by verifying the behavior of these gates through simulation results and synthesis diagrams. This hands-on experiment deepened your understanding of gate-level design and the practical use of Verilog for hardware description, preparing you for more complex FPGA-based digital circuit design tasks.

POST LAB QUESTIONS.

1. What do you mean by constraint file in VIVADO? What is the significance of it?
A constraint file in Vivado, commonly an XDC file, is used to define the physical connections and timing requirements of your digital design on an FPGA. It maps the logical signals from your Verilog or VHDL code to the actual physical pins of the FPGA chip and sets important parameters like clock frequencies and I/O voltage standards. This file is significant because it ensures that the design functions correctly when programmed onto hardware by making sure inputs and outputs are assigned properly and timing constraints are met. Without an accurate constraint file, the FPGA may not operate as intended, or worse, hardware damage could occur.
2. What is the difference between .v & .tb file in VIVADO?
The difference between a .v file and a .tb file in Vivado lies mainly in their purpose and usage. The .v file contains the actual Verilog source code of your digital design—it defines the modules, their inputs and outputs, and the internal logic that will be synthesized and implemented on the FPGA. In contrast, the .tb file, or testbench file, is created solely for simulation purposes; it is used to verify that your design behaves as expected by applying test inputs and observing outputs. The testbench does not get synthesized or programmed onto the FPGA; it is simply a temporary environment to test the design during simulation. Thus, while the .v file represents the final hardware logic, the .tb file supports design verification before implementation.

Department of Electronics and Telecommunication Engineering
(NBA ACCREDITED)
Digital System Design Laboratory
Academic Year 2025-2026, SEM-5, Odd Semester

ASSESSMENT RECORD.

Practical Performance [Model/Code, Results] (5 Marks)	Presentation/ Learning Outcome [Observation, Conclusion] (5 Marks)	Total (10 Marks)	Sign